Conf.h: The JQuery of Lua

Hugo Coto
hugo.coto@outlook.com

***Abstract*—In this paper we present a new library to read Lua code from C. The goal is to make Lua configuration files easy to use in our projects. That's why it ships as a single file with a small public API that does what you would expect. The first thing we had in mind when making decisions around this library was to help the user (*i.e., the developer*) to have a nice time and not suffer over such a secondary task: reading user configurations.**

***Index Terms*—C, Library, Lua, Configuration files**

I. Introduction

Reading configuration files in C is a pain. The Lua [1] standard approach involves manually calling a stateful API, managing a stack, writing verbose type-checking code, and keeping track of what is pushed and popped. Existing data-serialisation formats like JSON, YAML, and TOML are static: they describe data but cannot compute it, lacking variables, arithmetic, or control flow.

Here we present a new library, `conf.h`[2]. Its purpose is to abstract the libLua boilerplate needed to read values from Lua files, with the aim of using Lua as the configuration language for projects of all sizes. This library targets C developers, but can be used in C++, Zig and other compiled languages easily. The main feature that differentiates this library from other alternatives is that, although it is portable C, it is easy to use and ergonomic.

Why Lua? If you use state-of-the-art Linux software (i.e., Hyprland, Neovim, WirePlumber) you will notice they all use Lua for their configurations. This is no coincidence. The language is *fast*, readable, and well structured: it merges simple types with tables and functions, combining the descriptiveness of data serialization formats like TOML, YAML, and INI with the expressiveness and scripting features of other languages like Python or Ruby. We want to say that we are not sponsored by the Lua project.

This paper presents the library itself, its design rationale, and performance benchmarks comparing conf.h against hand-written code using the Lua C API. Section 2 surveys related work. Section 3 details the design and implementation. Section 4 presents performance measurements. Section 5 concludes.

II. Related Work

In this section we are going to show a few libraries that parse Lua for different programming languages, or can be used to parse other formats designed for configuration files. We have to say that we wrote our library before checking if someone had done it before. Not surprisingly, there are a lot of different alternatives, but none of them were written by us. The following libraries are well known options depending on your needs.

A. liblua

The official Lua C library[3] provides the stack-based C API for embedding Lua. [4] It is the foundation upon which conf.h is built, exposing the full Lua runtime: state creation, table access, registry references, and error handling. While powerful and complete, its API is verbose: reading a single table field requires multiple calls to push the key, fetch the value, and type-check the result. This is the alternative that we are going to use to compare performance.

B. LuaJIT

A Just-In-Time compiler for Lua, offering a drop-in replacement for the standard Lua interpreter with significantly higher performance. [5] Its FFI (Foreign Function Interface) allows C bindings to be declared directly from Lua code, making it a popular choice for both scripting and high-performance use cases. LuaJIT's C API is largely compatible with the standard Lua C API. Because conf.h uses the standard Lua C API, it runs on both PUC Lua and LuaJIT without modifications.

C. lupa

A Python binding for LuaJIT[6] that allows executing Lua code from Python. It targets Python developers who want to leverage Lua's sandboxed execution model or LuaJIT's JIT compiler for performance-critical numeric code. It focuses on interoperability between Python and Lua objects rather than configuration parsing. Unlike conf.h, lupa focuses on runtime interoperation between Python and Lua, not on reading configuration files from C.

D. slpp

A Lua source-code parser written in pure Python. [7] It performs syntactic analysis of Lua files without requiring the Lua runtime, making it suitable for linting, static analysis, or translation tools. Unlike conf.h, slpp performs purely static analysis without executing Lua, so it cannot resolve dynamic expressions or function calls at runtime.

E. sol2

A modern C++ wrapper for the Lua C API that uses template metaprogramming to bind complex C++ types to Lua. [8] It provides automatic type conversion, function overloading, and comprehensive error handling. Unlike conf.h,

sol2 is designed for binding C++ types to expose them to Lua scripts, whereas conf.h focuses on reading Lua configuration files into C variables.

F. mlua

Safe Rust bindings for the Lua C API. [9] mlua leverages Rust's ownership model to prevent memory safety violations when interacting with Lua's garbage collector, and supports both Lua 5.4 and LuaJIT. It is the standard choice for Rust projects that need Lua integration. Unlike conf.h, mlua provides safe async bindings with serde serialisation support for embedding Lua in Rust, whereas conf.h targets reading Lua configuration files into C with minimal code.

G. libucl

A universal configuration library written in C that parses nginx-like configuration files, JSON, and a YAML subset. [10] Unlike conf.h, which limits configuration files to Lua syntax, libucl is format-agnostic and optimised for server configuration. It does not support Lua's control flow, variables, or expressions.

H. TOML

A minimal configuration file format designed for unambiguous mapping to hash tables. [11] TOML files are easy for humans to read and for machines to parse, but the format lacks Lua's expressiveness: no variables, arithmetic expressions, or procedural logic.

I. YAML

A human-readable data serialisation format widely used for configuration. [12] YAML supports lists, dictionaries, anchors, and multi-document files, with parsers available in virtually every language. Like TOML, it is a static data format and does not provide the dynamic computation capabilities that Lua-based configuration offers.

J. Python/C API

The official C API for embedding the Python interpreter in C/C++ applications. [13] Python's rich standard library makes it a powerful configuration language, but its embedding footprint is orders of magnitude larger than Lua's (a minimal Python interpreter is several megabytes) making Lua a more practical choice for lightweight and fast configuration needs.

III. Design & Implementation

This library is designed around three main concepts: user happiness, portability and usability. To make it easy to integrate, we opted to distribute it as a single file, following stb [14] style; this way users can use it just by including the header as described at the top of the source code. It can also be compiled to object, shared or dynamic library as well, without any difficulty.

The public API was designed to be simple and follow a well-known structure. We opted for `open` and `close` functions to initialize and deinitialize the handler, and simple getters. We use an opaque handler to avoid adding complexity to the API. To use this opaque handler the user just needs to declare it, pass it by reference to `open` and then use (by value) as the first value in every other function from our library.

We decided to use the prefix `Conf_` to namespace the library. We capitalize it following raylib's naming convention, just because we love Ramon's work [15].

```c
Conf conf;
Conf_open(&conf, "init.lua");
...
Conf_close(conf);
```

Listing 1. Open/Close API.

The getters are what make this library unique. As C developers, we are accustomed to verbose APIs, something we do not like. We wanted to create a set of functions that can be used on their own with minimal additional code in the common circumstances. After trying different approaches, we chose three parameters, plus optional variadic parameters. The first argument is the opaque handler, the second is a pointer to the output variable, and the third (and consecutive) parameters form the field path.

As all of ricers know (i.e., people that enjoy to play with configuration files), Lua syntax is highly adopted because it allows developers to nest values inside tables. Inside these tables you can find other tables or literal values. Keep in mind that we read already-run Lua files, so functions have already been called and variables expanded. We define the path from the root of the main Lua file to the value the user wants to read, represented as a string: for a global value use `value`; for a nested one, use `table.value`.

You might be asking why we want variadic args. That's because Lua allows lists. You can read a list by using indexes in your path: `table.%d.value, i`. The format is the same that printf uses. You can use numeric indexes, but also `%s` for field names. This is highly ergonomic because you do not have to handle state, outer table references, or use any iterators. The only problem is the overhead of building the path on each call.

Lua's type system is very simple and close to C. That's why our getters support int (`int`), num (`double`), str (`char*`), and bool (`int`). With the same signature we also provide `len`, which reads the number of values in a list. Remember that Lua indexes start at 1.

```c
Conf_get_num (Conf, double *      , const char *, ...);
Conf_get_int (Conf, int *         , const char *, ...);
Conf_get_str (Conf, const char **, const char *, ...);
Conf_get_bool(Conf, int *         , const char *, ...);
Conf_get_len (Conf, int *         , const char *, ...);
```

Listing 2. Get API.

```lua
Table = { foo = { bar = 1 } }
List  = { { x = 10 }, { x = 20 } }
```

```c
Conf_get_int(conf, &val, "Table.foo.bar");
Conf_get_int(conf, &val, "List.%d.x", i);
Conf_get_str(conf, &str, "List.1.%s", field);
```

Listing 3. Example snippets.

As any Linux API developer may expect, all these functions return an int. This is the error code. If the return value is CONF_OK (0) / false the call succeeds. Otherwise, it may return other non-zero value defined in the Conf_Error enum. You have to refer to the specific function documentation to see what the possible values are and what is their meaning. To handle errors, you can choose the way you prefer, from insert it into an `assert(!Conf_ )` to a `if(Conf_ () != CONF_OK)` or the outstanding `switch (Conf_ ()) { case CONF_OK: ... }`.

We want to say that the API is explained in detail in the same source code. We recommend users to read all of it before using the API. (I know noone is going to do that).

IV. Performance

We are going to discuss the impact in speed of switching to this library. It is not designed with performance in mind, although we try to keep it as fast as possible. We chose liblua [4] as the reference point to measure the speed loss, because it is the official API and the backend of our library.

We are using conf.h version `1.0` and liblua version `5.5`. We use the same C program to measure the time for each test. On one hand, we are going to read a hundred values at a fixed nesting depth within the same table. On the other hand, we are going to read values at different nesting levels inside the same tables. These two points are where our library lacks in performance, because it does not cache any data. As our library uses the liblua API directly to extract the values to C variables, we only test int values as using other types does not impact on performance.

```c
Conf_get_int(conf, &val, "Number");
Conf_get_int(conf, &val, "Table.number");
Conf_get_int(conf, &val, "Table.table.number");
Conf_get_int(conf, &val, "Table.table.table.number");
```

Listing 4. `conf.h` snippet to test reads at different nesting levels.

```c
for (int i = 0; i < 1000; i++) {
    Conf_get_int(conf, &val, "Table.table.table.
%d.number", i)
}
```

Listing 5. `conf.h` snippet to test reads at the same nesting level.

Figure 1 shows the average time to access a single integer from a 9-levels-deep table using a path string (conf.h) compared to a pre-resolved registry reference (Lua C API). The cached reference is an order of magnitude faster, since conf.h must re-parse the full dot-separated path and walk the hash table on every call.

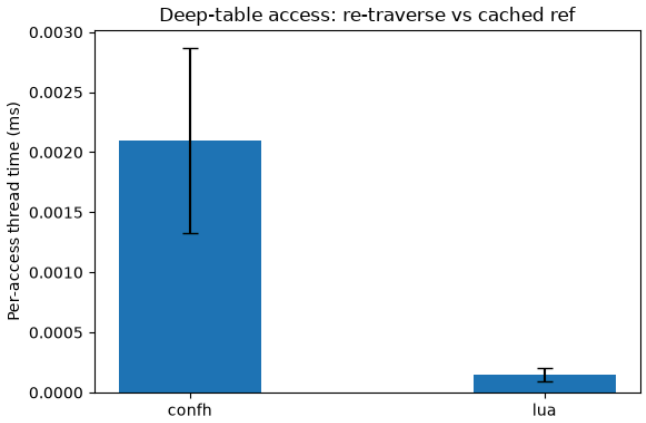

Fig. 1. Deep-table access time: re-traverse (conf.h) vs cached ref (Lua C API)

To understand how the cost scales with nesting depth, Figure 2 measures access time at depths 0 (top-level key) through 9. Conf.h's cost grows roughly linearly (≈ 0.2 µs per level), reflecting the path-parsing and hash-lookup overhead of each `table` hop. Lua, using a single registry reference per depth, stays near constant (≈ 0.1 µs).

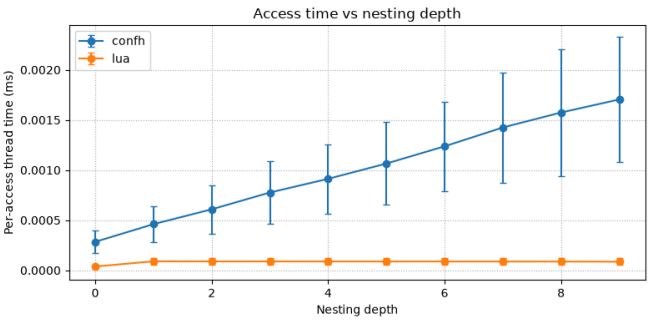

Fig. 2. Access time per nesting depth for conf.h and the Lua C API

Figure 3 plots the ratio of Lua time to conf.h time per depth. A ratio below 1 means Lua is faster. The gap widens with depth, confirming that string-path traversal is the dominant cost for conf.h.

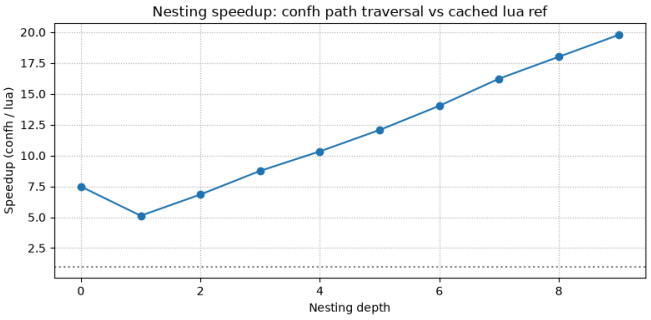

Fig. 3. Nesting speedup ratio of conf.h over the Lua C API

Finally, Figure 4 shows the per-trial speedup for the deep-table loop. Each dot is one of the 1000 trials. The ratio fluctuates slightly due to scheduler noise but stays consistently

below 1, confirming that Lua's cached-reference approach dominates across all runs.

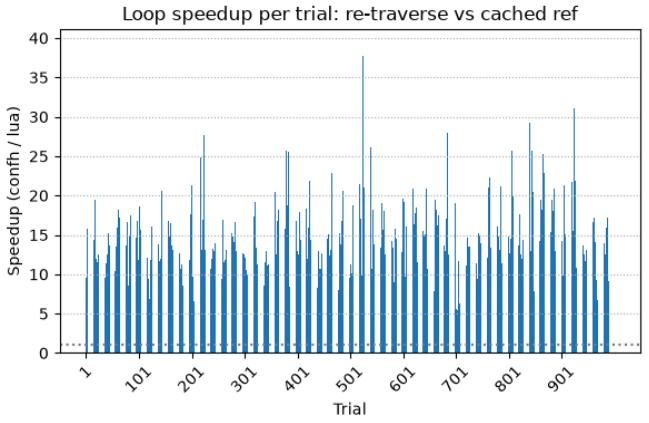

Fig. 4. Loop speedup ratio per trial

V. Conclusion & Future Work

Looking back at the design, the three pillars held up. The opaque handler keeps the API simple by hiding the Lua state, the user just declares a `Conf`, opens it, and reads values. The printf-style paths turned out more ergonomic than expected: `%d` and `%s` make dynamic table access painless, and no one has to touch the Lua stack. The main tradeoff is between ergonomics and performance, and for a library that runs once at startup, we stand by the choice.

The performance measurements show that conf.h's string-based path API is 2 to 17 times slower than equivalent hand-written Lua C code using cached registry references. This gap is expected: conf.h prioritises developer ergonomics over raw speed, trading parse-once semantics for a convenient dot-separated interface.

To put the numbers in perspective, consider a midsize configuration with 200 keys, averaging a depth of 3. From Figure 2, conf.h takes ≈ 0.8 µs per access at that depth, while Lua takes ≈ 0.1 µs. The total startup cost is:

$$\begin{aligned} \text{conf.h: } 200 \times 0.8\ \mu\text{s} &= 160\ \mu\text{s} \\ \text{Lua: } 200 \times 0.1\ \mu\text{s} &= 20\ \mu\text{s} \end{aligned} \tag{1}$$

The absolute difference is 140 µs, less than the time to render a single frame at 60 FPS (16.7 ms). For a read-once-at-startup workload the overhead is undetectable to the user.

What conf.h buys in return is conciseness. Reading a single key with conf.h is one line of C:

```c
Conf_get_int(conf, &val, "server.port");
```

The equivalent Lua C API code requires a global lookup, a table traversal, type checking, and stack management: about 8 to 10 lines per value, multiplied by 200 keys yields roughly 1,600 to 2,000 lines of boilerplate. Conf.h reduces this to 200 lines, making configuration parsing essentially free in both maintenance cost and runtime overhead.

As future work, we plan to add an internal cache that remembers the registry reference for each resolved path. No new functions, no changes to the API, it just gets faster without anyone noticing.

References

[1] PUC-Rio, "Lua: About Lua." [Online]. Available: https://www.lua.org/about.html

[2] H. Coto, "conf.h: Lua Configuration from C." [Online]. Available: https://github.com/hugoocoto/conf.h

[3] PUC-Rio, "Lua 5.4 Reference Manual." [Online]. Available: https://www.lua.org/manual/5.4/manual.html

[4] PUC-Rio, "Lua 5.4: download." [Online]. Available: https://www.lua.org/download.html

[5] M. Pall and others, "LuaJIT." [Online]. Available: https://luajit.org/luajit.html

[6] S. Behnel and others, "lupa: LuaJIT wrapper for Python." [Online]. Available: https://github.com/scoder/lupa

[7] Anthony and others, "slpp: Lua parser for Python." [Online]. Available: https://github.com/SirAnthony/slpp

[8] ThePhD, "sol2: C++ binding for Lua." [Online]. Available: https://github.com/ThePhD/sol2

[9] mlua-rs contributors, "mlua: Rust bindings for Lua." [Online]. Available: https://github.com/mlua-rs/mlua

[10] V. Stakhov, "libucl: Universal Config Library." [Online]. Available: https://github.com/vstakhov/libucl

[11] T. Preston-Werner and others, "TOML: Tom's Obvious Minimal Language." [Online]. Available: https://toml.io/en/

[12] O. Ben-Kiki, C. Evans, and I. döt Net, "YAML Ain't Markup Language (YAML^{tm}) Version 1.2." [Online]. Available: https://yaml.org/

[13] Python Software Foundation, "Python/C API Reference Manual." [Online]. Available: https://docs.python.org/3/c-api/index.html

[14] S. Barrett, "stb: single-file public domain libraries for C/C++." [Online]. Available: https://github.com/nothings/stb

[15] R. Santamaria and others, "raylib: A simple and easy-to-use library to enjoy videogames programming." [Online]. Available: https://www.raylib.com/